Github with code - https://github.com/benpurdum/BIBIFI_Secure_Programming

Guest role:

- move between rooms
- view authorized information
    - personal and art gallery stuff
- login/logout

Employee role:

- move between rooms
- view authorized information
    - personal, art gallery stuff
- make/record events in the gallery
- see # of people in rooms
- login/logout

Admin role:

- manage everything
    - users, roles, events
- see # of people in rooms
- login/logout

Functional requirements:

- Person enters the gallery.
- Person leaves the gallery.
- Person enters a room.
- Person leaves a room.
- View current gallery/room occupancy.
- Search and query historical events.
- Administrative functions

Explicit security requirements:

- Secure password storage
- Authentication
- Authorization
- Input validation

- Output encoding
- SQL injection prevention
- Cross-Site Scripting (XSS)
- Broken access control / IDOR
- Session/token security
- Information disclosure
- Secure error handling
- Secrets management
- Audit logging
- Security misconfiguration

Week 5 – Secure Design and Threat Modeling

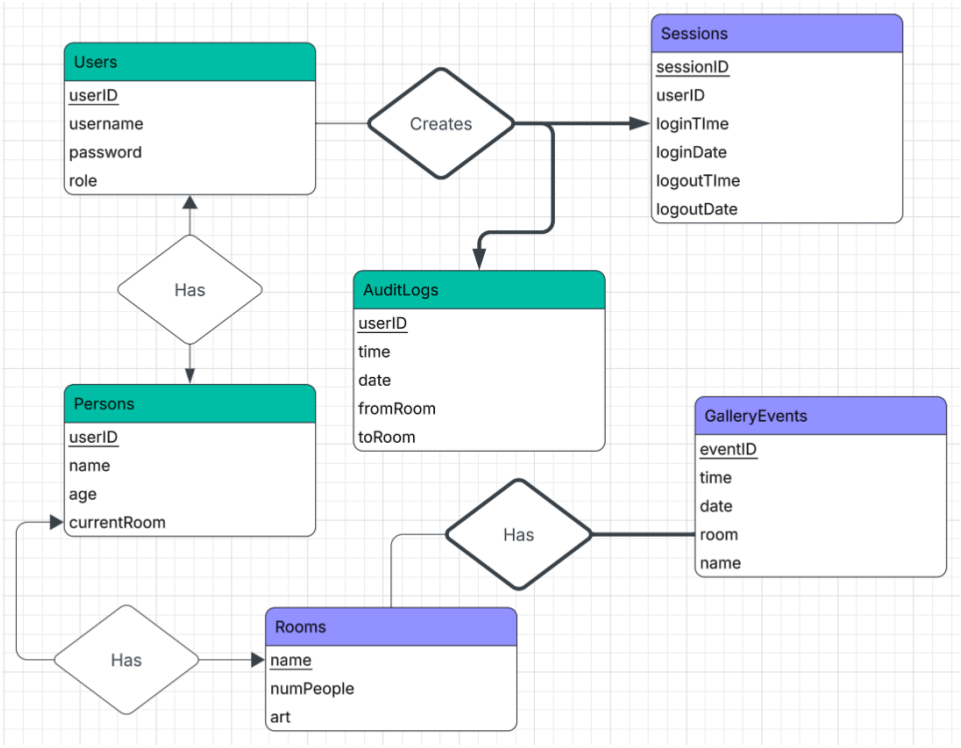
Architecture is going to be flask through docker with an awesome database file

Stride Model

Threat	Description	Attack Scenario	Impact	Mitigation
Spoofing	Can an attacker impersonate a user?	Attacker impersonates an employee/admin to steal data and other bad things	Private data can become compromised.	Secure password hashing and strong password requirements
Tampering	Can data be modified without authorization?	Attacker changes admin password to lock them out of the account	Data loses integrity after being modified.	Check user roles every time data is modified
Repudiation	Can actions be denied or hidden?	An attacker modifies data without authorization and is able to do so without being traced	Attackers may not be identified, making it difficult to protect from future attacks	Implement a logging system to keep track of user actions
Information Disclosure	Can sensitive information be exposed?	An attacker finds a page unintentionally exposing private data	Personal data of users may become exposed and used maliciously	Enforce authorization when necessary and limit data able to be

				accessed
Denial of Service	Can the application be made unavailable?	Attacker uses a DDOS attack on system to make services unavailable for use.	The website may become slower or unavailable for legitimate users	Restrict the request rate and filter requests
Elevation of Privilege	Can a user gain unauthorized permissions?	An attacker exploits a vulnerability in authorization to assign themselves as admin	Regular users may be granted permissions originally allowed only to admin users	Implement role based access control instead of granting privileges based on individual accounts

ER Diagram:



Data Flow Diagram:

